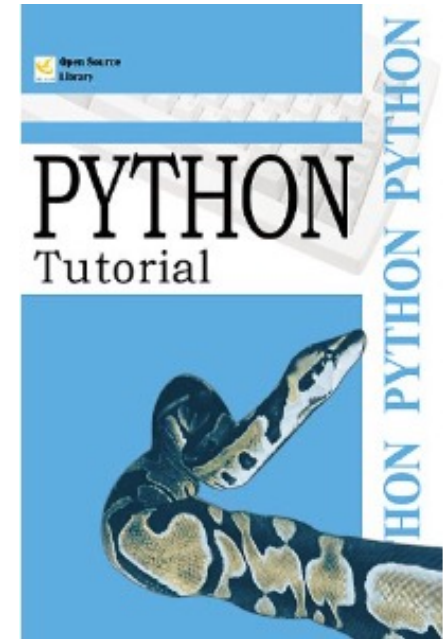


SQLite



Dra. M^a Dolores Rodríguez Moreno



Universidad
de Alcalá



ISG

Objectives

Specific Objectives

- Understanding and using SQLite databases (DDBB) from the terminal and from Python

Source

- <https://docs.python.org/3/library/sqlite3.html>
- <https://ellibrodepython.com/>
- Python Tutorial (paperback). Guido van Rossum (2012)

Outline

- Introduction
- Create DDBB from CSV
- Create DDBB with a program

SQLite

- A lightweight, file-based database system (DDBB)
- Stores the whole database in a single binary file & combines features of DDBB and file handling
- Relational DDBB: data is stored in **binary** format and queried with SQL
- Self-contained, serverless, and highly portable
- Suitable for small to medium-sized applications
- 4 operations for persistent storage: Create, Read, Update, Delete (CRUD)
- Why Use SQLite in Python?
 - Built-in support via the sqlite3 module
 - No setup or installation required
 - Ideal for prototyping and development

Outline

- Introduction
- **Create DDBB from CSV**
- Create DDBB with a program



Create DDBB from CSV (interactive)

- Open a terminal and run: `sqlite3`
- `.mode csv`
- `.import 'file_name' name_table`
- `.schema`: to see the table
- `SELECT * FROM name_table;`
- `.save file.db`: save the database (in binary) into a file
- `.quit`: to leave the environment

Create DDBB from CSV

```
sqlite> .quit
○ (base) MacBook-Pro-de-Maria-2:Files mariadr-moreno$ sqlite3
SQLite version 3.32.3 2020-06-18 14:16:19
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .mode csv
sqlite> .import 'output.csv' prueba
sqlite> SELECT Name FROM prueba;
Alice
Bob
sqlite> .save prueba.db
sqlite> .quit
```

Outline

- Introduction
- Create DDBB from CSV
- **Create DDBB with a program**

Create a DDBB with a program

1. Database Connection
2. Creating Tables
3. Inserting Data
4. Reading Data
5. Updating Data
6. Deleting Data
7. Closing the Connection

Connection

- Import the *sqlite3* module
- Connect to a database file (creates one if it doesn't exist)

Example: connection

```
import sqlite3  
  
connection = sqlite3.connect('prueba.db')
```

Create Table

- Create a cursor object using *connection.cursor()*
- Use *execute()* method to run SQL commands (add IF NOT EXISTS to prevent an error if the table exists)
- Then, commit changes in the DDBB using *connection.commit()*

Example: create table

```
cursor = connection.cursor()

cursor.execute('''

    CREATE TABLE IF NOT EXISTS prueba (

        Name TEXT,

        Age TEXT, # or we can define it as INTEGER

        Country TEXT

    )

''')

connection.commit()
```

Insert Data

- Use INSERT INTO to add a row to your table

Example: insert data

```
row = ("Charlie", "32", "Canada")    # one tuple

cursor.execute('''

    INSERT INTO prueba (Name, Age, Country)

    VALUES (?, ?, ?)

''', row)    # [ADD] pass the tuple

connection.commit()
```

Insert Data

- Use INSERT INTO to add rows to your table

Example: insert data

```
people = [("Alice", 30, "USA"), ("Bob", 25, "UK"), ("Charlie", 32, "Canada")]

cursor.executemany('''

    INSERT INTO prueba (Name, Age, Country)

    VALUES (?, ?, ?)

''', people)

connection.commit()
```

Reading Data

- Use *fetchall()* or *fetchone()* to retrieve query results

Example: read data

```
cursor.execute('SELECT * FROM prueba')

rows = cursor.fetchall()

for row in rows:

    print(row)
```

Update data

- Modify existing records with UPDATE

Example: update

```
cursor.execute('''  
  
    UPDATE prueba  
  
    SET Name = ?  
  
    WHERE Name = ?  
  
''', ('Alicia', 'Alice'))  
  
connection.commit()
```

Delete data

- Remove records with DELETE

Example: delete

```
cursor.execute('''  
  
    DELETE FROM prueba  
  
    WHERE Name = ?  
  
''', ('Bob',))  
  
connection.commit()
```

Closing connection

- Always close the connection when done to free resources

Example: close

```
connection.close()
```

- Visualize your DDBB: <https://sqlitebrowser.org/>